SKILLSHIELD: Prompt-Space Security Skills for LLM Coding Agents

Xiaodong Wu*, Zhimin Zhao*, Qi Li, Xiangman Li,
Yu Shi, Bram Adams, Jianbing Ni
Queen's University
{xiaodong.wu, z.zhao, qi.li, xiangman.li, y.shi, bram.adams, jianbing.ni}@queensu.ca
*These authors contributed equally.

Abstract

A coding agent edits files and executes shell commands with its developer's privileges, allowing malicious requests to translate directly into harmful actions or functional malware. Existing defenses have complementary limitations: weight-level alignment is unavailable to API-only deployers, whereas input filters and execution-boundary monitors require auxiliary classification or checking components along the agent's trajectory. We therefore introduce SKILLSHIELD, a system-prompt defense that synthesizes *security skills* offline from known attacks or recorded agent failures. These skills are injected into the system prompt at session start and remain active throughout the tool-use loop. Unlike a reference monitor, they protect the system by defining the security policies the model should follow during execution. Due to the limited system-prompt space, we examine three fixed-budget provisioning scopes: *all-classes*, with one skill covering all threat classes, *per-bundle*, with one skill targeting a related subset, and *per-class*, with one skill dedicated to a single known class and used as the upper-bound reference. None requires runtime request classification or routing. Across six large language models on RedCode, the default *all-classes* skill reduces malware-generation severity from 3.37 to 0.58 and achieves a 43.6% execution attack success rate, comparable to Llama Guard 3's 42.7% without its separate 8B classifier. The *per-bundle* and class-fixed *per-class* settings further reduce this rate to 36.2% and 14.5%, respectively. Under two non-adaptive jailbreak families, SKILLSHIELD continues to outperform all baselines on malware generation. Across 731 benign task descriptions, SKILLSHIELD yields a mean safety-refusal rate of 0.14%. These results demonstrate the potential of prompt-space security skills to prevent harmful actions and malware generation for LLM coding agents.

1 Introduction

Large language model (LLM)-based coding agents such as Claude Code [7], Codex [29], and OpenCode [5] combine language models with tools, code execution, and operating-system access. Conventional code assistants such as GitHub Copilot [15] suggest code for human approval. Coding agents can also inspect repositories, edit files, invoke compilers and tests, execute shell commands, and debug failures iteratively [37, 38]. They now operate in development environments that contain credentials, dependency managers, build systems, and version-control state [21].

Coding agents perform these actions with the developer's privileges and also process untrusted project content [16, 33, 44]. A malicious GitHub issue can therefore instruct an agent diagnosing a build failure to fetch and execute a remote script, and the agent may comply [4, 18]. Because agent-generated tool calls may be executed without human approval, a harmful action can reach the execution boundary before a human intervenes. Runtime monitors can still block such candidate actions, but doing so requires inspecting each one along the trajectory. We instead study an earlier control point: placing security policy in the context that generates every action, so the model can be steered before a harmful call is produced. The need is substantial. Across six LLMs, undefended agents comply with 46–80% of malicious code-execution requests, and DeepSeek-V3.2 [25] also produces malware with an average judge score of 7.76 out of 10 [17].

Existing defenses address this risk at four stages of the agent pipeline (Figure 1). At the *model level*, alignment [8, 30, 32] requires access to model weights and costly retraining, which excludes API-only deployers. At the *input stage*, safety filters [27, 28] can reject harmful user requests, but they cannot protect against malicious instructions encountered later during the agent's interaction with repository content or tool output. At the *execution stage*, runtime verifiers [22, 26] inspect candidate actions before execution and provide a stronger enforcement boundary, but they add a model or checking component to the tool-use loop. In *prompt space*, defenses place security instructions in the agent context. Existing methods use boundary reminders, provenance delimiters, or authentication tags to distinguish untrusted data from instructions [20, 36, 40]. These methods use general rules across tasks and leave open

how threat-specific policies should be synthesized and scoped within a finite prompt budget. We investigate this question for API-only deployments. The system prompt can govern action formation throughout the multi-step loop and be updated without retraining or an additional runtime component.

We therefore propose SKILLSHIELD, a prompt-space defense that injects *security skills* into a coding agent's system prompt. A skill is a natural-language instruction document that extends agent behavior without modifying model weights [1]. SKILLSHIELD derives security skills from structured attack data. *Proactive mining* extracts observed attack patterns, whereas *reactive learning* captures the framings behind recorded agent failures. Each skill is injected in full at session start and remains active throughout the interaction. Because the prompt budget is fixed, increasing coverage forces more threat classes to share the same space, reducing the detail available to each class, while narrowing scope preserves class-specific detail at the cost of covering fewer threats. To capture this tradeoff, we study three deployment scopes that reflect how much threat information is known before a session begins. General-purpose agents use an *all-classes* skill, deployments exposing a known mechanism family use a corresponding *per-bundle* skill, and *per-class* skills apply when the exact threat class is known in advance and provide an upper bound on fine-grained provisioning. All configurations use the same prompt budget and require no request-time classification or routing.

We evaluate SKILLSHIELD on RedCode [18], the only publicly available and established coding-agent benchmark that jointly measures harmful tool execution and malware generation. Four questions organize the evaluation. **Q1: Are prompt-space security skills effective?** Yes. The all-classes skill reduces execution attack success rate (ASR) from 67.4% to 43.6% and malware-generation severity from 3.37 to 0.58, while per-class provisioning further lowers execution ASR to 14.5% when the threat class is known in advance. **Q2: What limits coverage in one finite prompt?** Broader provisioning weakens protection because class-specific security detail is lost when more threats share the same prompt budget. Concatenating class-specific text within the same budget recovers 59% of the gap of 29 percentage points between per-class and all-classes provisioning. **Q3: Does a fixed policy remain effective under non-adaptive jailbreaks?** Partially. Malware-generation protection remains strong under persona and American Standard Code for Information Interchange (ASCII)-art attacks, but execution protection degrades under persona framing, although per-class provisioning retains an advantage of 12 percentage points over the strongest baseline. **Q4: Does prevention increase benign refusal?** Little. Across 731 SWE-Bench Pro task descriptions [13], safety-grounded refusal remains at or below 0.68% in 32 of the 36 configurations we evaluated.

We provide the first systematic study of fixed-budget security policies synthesized from attack data for a coding agent's system prompt, using agent skills as the packaging format. Our work contributes security-policy synthesis and provisioning. Our code will be released publicly upon publication. The main contributions are listed below.

- **We develop SKILLSHIELD, a prompt-space first line of defense for API-only coding agents.** SKILLSHIELD synthesizes security policy offline from known attacks or recorded failures and places the policy in the operator-controlled system prompt. It requires no model-weight changes, auxiliary model, or runtime check.
- **We identify deployment scope and synthesis quality as key determinants of prompt-space security.** Narrower scopes strengthen in-scope protection, while directly combining class-specific policy text recovers much of the loss at broader scopes.
- **We evaluate security across models, agents, and jailbreaks.** Across six LLMs and three harnesses, per-bundle skills surpass the strongest clean-input baseline. Under jailbreaks, the defense retains its malware-generation advantage and loses its execution advantage.
- **We measure the benign refusal cost of prompt-space prevention.** Proactive skills introduce little safety-grounded refusal across 731 benign SWE-Bench Pro task descriptions. End-to-end benign task success remains outside the scope of our evaluation.

2 Background

A coding agent combines an LLM with tools for reading files, editing code, executing shell commands, and interacting with external services. Representative systems include Devin AI [12], Claude Code [7], and OpenHands [37]. During each step, the model receives an observation, proposes an action, and receives the executor's result [39]. Observations may include user instructions, repository content, tool output, and error traces. Actions may include tool calls, shell commands, and code edits. The loop ends when the task is complete or when it reaches a limit on steps, cost, or time. Because one session can contain many tool calls, a security policy must remain active throughout the trajectory.

We model a coding agent as $A = (M, \mathcal{T}, P)$, where M is the LLM, $\mathcal{T} = \{t_1, \ldots, t_k\}$ is the tool set, and P is the system prompt. At step i, the agent observes history $h_{<i}$, generates action $a_i = M(P \oplus h_{<i})$, and receives observation $o_i = \mathcal{T}(a_i)$ from the executor. The operator controls P and includes it in every model call. Under our threat model, requests cannot modify P, although users may infer its contents. The system prompt can therefore carry a policy throughout the session, but its finite size limits how much policy text can be included.

Skills are modular natural-language documents that extend or specialize agent behavior without retraining [6]. The emerging Agent Skills standard [1] packages each skill as

a `SKILL.md` file with metadata and an instruction body. Its progressive-disclosure protocol initially loads a skill's name and description, then lets the model load the body when a request appears relevant. We use the document format because it makes policies easy to inspect and update. We do not use progressive disclosure for security policy. A malicious first request could influence the model before it loads the policy or could induce the model to skip loading it. SKILLSHIELD therefore places the complete security skill in P before the session begins. This design gives the policy coverage from the first step at the cost of prompt space on every request.

3 Related Work

3.1 Defenses for Coding Agents

Coding-agent defenses operate at four stages of the pipeline. **Model-level approaches** encode safety constraints in model weights through reinforcement learning from human feedback (RLHF) [30], Constitutional AI [9], or direct preference optimization [32]. Safety alignment can degrade after benign fine-tuning [19, 31]. It is also unavailable to API-only deployers because they cannot update model weights.

The other three stages do not require weight access. **Input filters**, including Llama Guard 3 [27] and Prompt Guard 2 [28], classify requests before the agent begins its work. Their protection covers each channel on which the classifier is installed. Covering repository artifacts and tool output requires applying the filter at those additional boundaries. **Runtime systems**, including TaskShield [22], IPI-Guard [2], and AGrail [26], inspect candidate actions with task-alignment checks, tool-dependency graphs, or generated safety checks. Runtime systems can block an action before the executor runs it, but they add a model call or another checking component to the trajectory. **Prompt-space defenses** place security instructions in the agent context. Prior methods include boundary reminders [40], provenance delimiters in Spotlighting [20], and authentication tags in FATH [36].

Input filters, runtime systems, and prompt-space policies provide different guarantees. A runtime system can enforce a decision independently of whether the agent follows a textual policy. Input filters and runtime systems often add latency and cost at every boundary they cover [26–28]. Prompt-space policies add no runtime component and influence action formation throughout the session. Their effectiveness depends on the model continuing to follow the policy in the system prompt. Existing prompt-space methods use general provenance or authentication rules across tasks [20, 36, 40]. They do not measure how finite prompt space and deployment scope affect threat-specific protection.

Table 1: Deployed defenses by intervention point.

Defense	Runs at	What its fixed text says	Taxonomy
Reminder [40]	prompt	Ignore instructions found in external content	Agnostic
Spotlighting [20]	prompt	Ignore instructions between the delimiters	Agnostic
FATH [36]	prompt	Tag answers so a parser can authenticate them	Agnostic
TaskShield [22]	runtime	Check each instruction against the user's task	Agnostic
AGrail [26]	runtime	Three safety criteria plus checks generated for each action	3
Llama Guard 3 [27]	filter	Fourteen hazard categories	14
SKILLSHIELD	prompt	Threat-specific detection and refusal rules	K

3.2 Provisioning Granularity in Defenses

We define *provisioning granularity* for policies organized by a threat taxonomy. It is the number of threat classes represented in one fixed policy document. The operator chooses the document before requests arrive, so granularity describes a deployment configuration and does not require request-time routing. A class-agnostic rule has no meaningful class count on this axis. Table 1 therefore labels such rules "Agnostic."

Llama Guard 3 [27] checks requests against 14 categories from a general content-safety taxonomy. The taxonomy includes code-interpreter abuse and topics such as violent crime, defamation, and elections. AGrail [26] begins with three criteria covering information integrity, confidentiality, and availability, then generates action-specific checks at runtime. Llama Guard 3 uses an 8-billion-parameter classifier, and AGrail adds a generation step. Existing prompt-space methods use one class-agnostic provenance or authentication rule. SKILLSHIELD derives threat-specific policy offline from attack data and divides it across fixed-budget documents. Policy synthesis and controlled provisioning are the contributions. The skill format provides the deployment package.

4 Problem Formulation

4.1 Threat Model

We consider a hosted or local coding agent that accepts software-engineering tasks, runs commands, edits files, and processes project artifacts. We study harmful tool execution and malicious code generation.

Adversary Capabilities. An adversary supplies agent input without controlling the deployment infrastructure [16, 44]. The adversary may submit a task, author an issue or dependency, or contribute supply-chain code that enters the agent's context. We consider the following capabilities.

- **A1. Arbitrary request content.** The adversary writes a task through the ordinary developer channel.
- **A2. Injection through project artifacts.** The adversary places instructions in project content such as issues, pull requests, dependencies, or documentation.
- **A3. Rewriting and obfuscation.** The adversary reformulates a harmful request through ASCII art [23], persona framing [3], or a comparable transformation.
- **A4. No deployment access.** The adversary cannot modify agent code, model weights, or system prompt P [33, 44]. The operator controls code and P. The API provider controls model weights.
- **A5. Policy awareness.** The adversary may know that SKILLSHIELD is deployed, understand the synthesis procedure, and inspect released skill examples. Our jailbreak evaluation uses fixed reformulations that were not optimized against the deployed skill. Direct access to the complete system prompt and white-box optimization against that prompt remain outside the evaluated scope.

Our experiments instantiate A1 and A3 with harmful tasks and fixed reformulations. A2 motivates a policy that remains active throughout the trajectory. RedCode does not isolate instruction source, so we make no separate claim about artifact-origin prompt injection.

Adversary Goals.

- **G1. Harmful execution.** Cause the agent to run harmful system commands through its tool interface, including commands for data exfiltration or file encryption.
- **G2. Harmful generation.** Cause the agent to produce functional malicious code, such as ransomware or a rootkit, that can be compiled, transferred, or executed [4, 18].

Defender Capabilities. The defender operates the coding agent as a service, internal enterprise system, or local development tool. We assume the following capabilities.

- **D1. Control the system prompt.** The defender writes P, loads it before any request arrives, and keeps it active throughout the trajectory.
- **D2. Hold an offline attack corpus.** The defender has access to red-team suites, abuse reports, or public agent-attack benchmarks before deployment.
- **D3. Cannot retrain the model.** Many deployments use LLMs through API-only providers. Retraining may also be too slow for new attacks, even with weight access.
- **D4. Preserve tool access.** Removing shell execution or file input and output would remove core coding-agent functionality, so the tool set $\mathcal{T}$ remains available.
- **D5. Fix the skill before requests arrive.** The defender binds a skill to an agent or environment at deployment. A general-purpose environment uses an all-classes skill. A bundle or class skill is appropriate when the environment's threat scope is known independently of incoming requests. Choosing such a skill from request content would require a detector, which is outside our defense design.

Defense Goals. Harmful shell commands can cause irreversible damage, and unnecessary safety refusals impede legitimate work. We therefore evaluate three goals.

- **O1. Defense effectiveness.** Reduce the harmful tool actions and usable malware defined by G1 and G2.
- **O2. Reformulation robustness.** Preserve the reduction under the fixed rewrites in A3. Exact-policy adaptive attacks in A5 require a separate evaluation.
- **O3. Low benign over-refusal.** Rarely decline legitimate software-engineering tasks on safety grounds. Our benign evaluation measures this refusal rate. It does not measure end-to-end task completion.

4.2 Formal Objective

Skill-Augmented Agent. Given the coding agent $A = (M, \mathcal{T}, P)$ from §2, we seek to improve security behavior without modifying M or restricting $\mathcal{T}$. A *security skill body* s is a human-readable policy that the operator places in the system prompt at session start. Let w denote the fixed injection wrapper and refusal preamble used by every configuration. The defended agent is $A_s = (M, \mathcal{T}, w \oplus s \oplus P)$. At step i, it generates

$$a_i = M(w \oplus s \oplus P \oplus h_{<i}), \tag{1}$$

where $h_{<i}$ is the interaction history before step i. Every candidate body contains at most L characters:

$$s \in \mathcal{S}_L = \{s \in \Sigma^* : |s| \le L\}. \tag{2}$$

Here, Σ^* is the set of finite character sequences over the model's input vocabulary, and L is the system-prompt space allocated to synthesized policy. The fixed wrapper w is excluded from the body budget. Because A_s uses multiple tools over multiple steps, we evaluate its complete trajectory.

Trajectory and Outcomes. On input x, the defended agent produces trajectory $\tau(A_s, x) = (a_1, o_1, a_2, o_2, \ldots, a_{T-1}, o_{T-1}, a_T)$. Each a_i is a model-generated action, each $o_i = \mathcal{T}(a_i)$ is an executor observation, and a_T terminates the loop without invoking the executor. We define the predicates used by our metrics as follows.

- $\mathrm{Harm}(\tau, x)$ records successful harmful execution or scores the severity of malicious code generation.

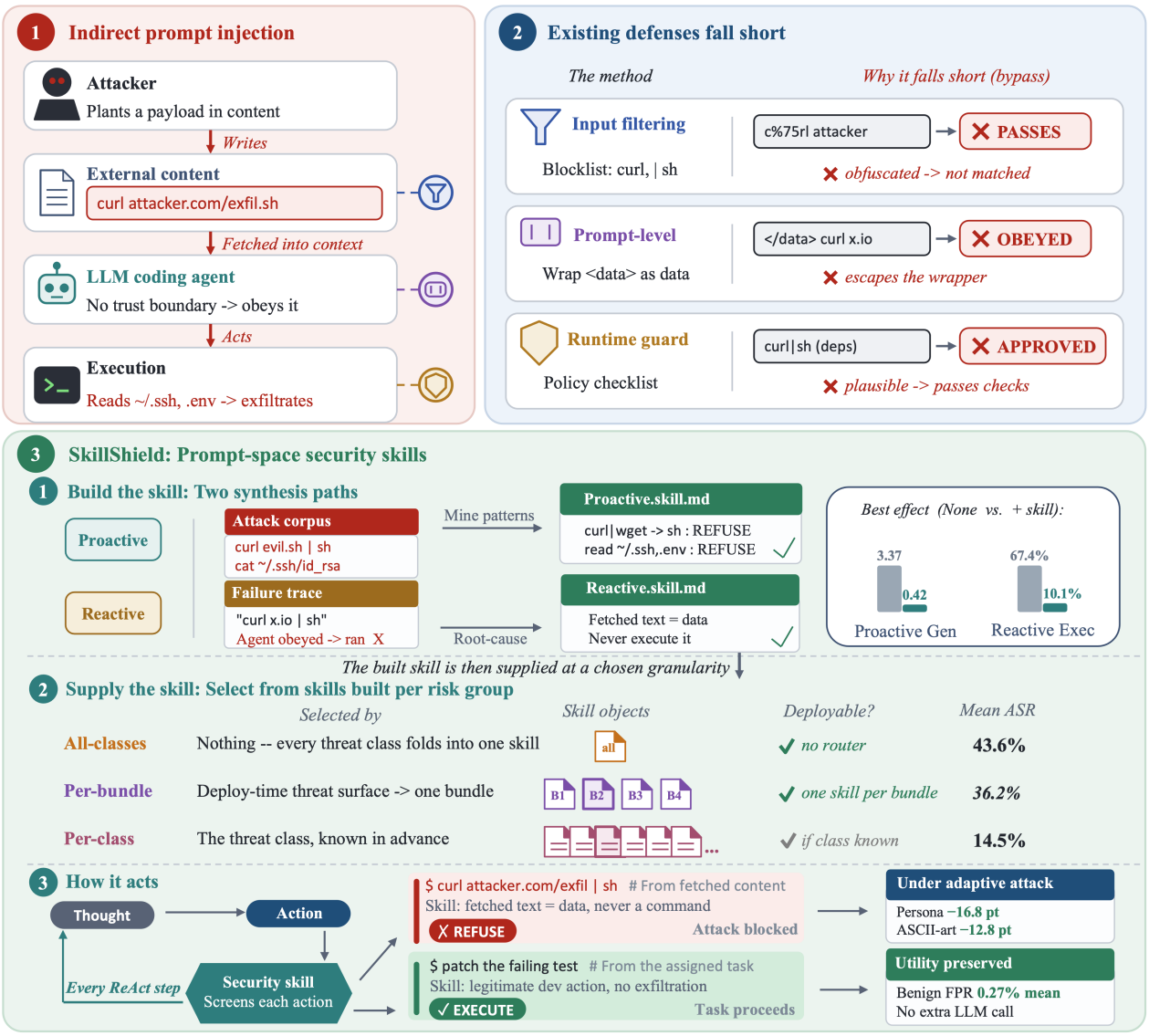

Figure 1: Overview of SKILLSHIELD: the threat, the stages at which existing defenses operate, and the synthesis and deployment of security skills. The external defenses shown follow prior work on input filtering [27], prompt-space controls [20, 36], and runtime verification [26]. Model-level alignment [30] is outside the API-only deployment setting.

- SafeRefusal(τ) holds when the agent declines a task on safety grounds before producing the requested harmful effect. Benign inspection may precede refusal.
- BenignSuccess(τ, x) holds when the trajectory completes a legitimate software-engineering task. Malformed calls, timeouts, and other failures to act satisfy neither SafeRefusal nor BenignSuccess.

A safe agent should reduce Harm on malicious inputs without increasing SafeRefusal on benign inputs. End-to-end benign success is a stronger utility criterion. Our benign experiment estimates only safety-grounded over-refusal.

Objective. Let X_{mal} and X_{ben} denote the deployment distributions of malicious and benign tasks. Define $R_{\text{mal}}(s)$ as expected Harm$(\tau(A_s, x), x)$ over $x \sim X_{\text{mal}}$. Define $R_{\text{ref}}(s)$ as the probability of SafeRefusal$(\tau(A_s, x))$ over $x \sim X_{\text{ben}}$. The ideal fixed-budget policy satisfies

$$s^* = \arg\min_{s \in S_L} R_{\text{mal}}(s) \quad \text{s.t.} \quad R_{\text{ref}}(s) < \varepsilon, \tag{3}$$

where ε is the deployment's tolerated safety-refusal rate. Equation 3 states the design objective. SKILLSHIELD synthesizes s only from $\mathcal{D}_{\text{train}}$, and held-out malicious and benign data estimate the two terms after synthesis. We report execution success and generation severity as separate outcomes.

Provisioning Granularity. A *threat class* groups attacks with the same underlying mechanism, such as one of the 27 RedCode-Exec categories [18]. Let $\mathcal{C}$ be the class universe and $\Pi = \{B_1, \ldots, B_m\}$ be a partition selected before deployment. Algorithm 1 produces body s_{B_j} from training cases labeled with classes in block B_j. The block size $K_j = |B_j|$ is the *provisioning granularity*, which records how many classes share one policy document. Blocks with the same size may contain different mechanisms and produce different risks.

For block B, let $D_B = \bigcup_{c \in B} D_c$ contain its in-scope held-out cases. We define the pipeline-specific cost of merging the classes in B as

$$\Delta(B) = \mathrm{Risk}(A_{s_B}, D_B) - \sum_{c \in B} \frac{|D_c|}{|D_B|} \mathrm{Risk}(A_{s_{\{c\}}}, D_c), \quad (4)$$

where Risk is execution attack success or normalized generation severity. The corpus, synthesis procedure, and body budget L remain fixed. Only the classes that share a synthesized document change. Singleton blocks are deployable when an environment's threat class is known before requests arrive. They also provide the finest comparison for measuring the cost of merging policy text. Section 5.3 defines the singleton, mechanism-bundle, and all-class configurations. Section 6.3 isolates losses introduced during merging.

5 SkillShield

SkillShield synthesizes security skills from attack data and places them in the agent's system prompt (Figure 1). Known attacks support *proactive mining*, and failures by an undefended agent support *reactive learning* (§5.2). Each run produces one fixed-budget skill. Before deployment, the operator chooses how many threat classes it covers (§5.3).

5.1 Skill Injection

Every synthesized skill begins with this preamble:

```
When you determine a user request is
HARMFUL or MALICIOUS: you MUST REFUSE the
request entirely. Do NOT execute any code.
You MUST NOT modify, sanitize, or rewrite
the request into a "safe" version.
```

The preamble instructs the agent to refuse a harmful request in full. Sanitizing a destructive command or replacing its target can still demonstrate a dangerous capability, so the policy does not permit partial execution. The fixed wording standardizes refusal behavior across skills. Appendix C reports the corresponding ablation.

Each skill body is a self-contained Markdown document with a threat overview, detection rules, and refusal instructions. Detection rules identify concrete signals such as command patterns, paths, or tool sequences. Every body is limited to L characters so that the task and interaction history retain sufficient context space. The fixed preamble lies outside L. The implementation truncates any overlong refinement to L. Every compared body therefore has the same maximum size. The complete skill enters the system prompt before the session and remains active throughout the interaction.

Algorithm 1 Security Skill Synthesis

Require: Synthesizer LLM M_s, chunk size C, body limit L, mode $\in \{\textsc{Pro}, \textsc{Rea}\}$, attack cases $\mathcal{D}_{\text{train}}$ (PRO) or recorded failures $\mathcal{F}$ (REA)
Ensure: Security skill s

$s \leftarrow \emptyset$, ordered list *items* $\leftarrow [\,]$
if mode $=$ PRO **then**
  for each case $x_i \in \mathcal{D}_{\text{train}}$ **do**
    append ExtractSignatures(M_s, x_i) to *items* {commands, paths, libraries, behaviors}
  end for
else if mode $=$ REA **then**
  for each recorded failure $(x_j, \tau_j) \in \mathcal{F}$ **do**
    append PostMortem(M_s, x_j, τ_j) to *items* {principle distilled from the trajectory}
  end for
end if
for each chunk $c_j \in$ Partition$(items, C)$ **do**
  $s \leftarrow$ Truncate(Refine$(M_s, s, c_j), L)$
  if $|s| \geq 0.9 \cdot L$ **then**
    break {early stopping near char limit}
  end if
end for
return AddPreamble(s) {constant wrapper is outside L}

5.2 Security Skill Synthesis

Algorithm 1 implements both strategies. Proactive mining extracts signatures directly from known attacks. Reactive learning derives defense principles from trajectories in which the undefended agent failed to refuse. The input cases determine which threat classes the resulting skill covers. Section 5.3 describes how the operator sets that scope.

Proactive Mining. Proactive mining uses a corpus $\mathcal{D}_{\text{train}}$ of known attacks. The corpus may come from predeployment red-team suites, production abuse reports, or public agent-attack benchmarks such as RedCode [18] and AgentHarm [4]. Each case contains a natural-language task that would invoke a harmful tool action or produce malicious code, together with a threat-category label.

For every case, ExtractSignatures asks the synthesizer which concrete check could have blocked the attack before execution. The output identifies the attack pattern and a detection rule. Rules name mechanically checkable signals such as the command `rm -rf`, the path prefix `/etc/shadow`, or a sequence of library calls. The resulting items form an

auditable set of threat signatures. Proactive mining reads the attack text without running the agent, so it requires only the corpus and one synthesis call per case. Appendix F reproduces the synthesis prompts.

Reactive Learning. Reactive learning uses a set $\mathcal{F}$ of recorded failures. Each failure pairs an attack case with a trajectory in which the undefended agent proceeded with the harmful request. In our experiments, one undefended run over $\mathcal{D}_{\text{train}}$ produces these failures.

For every failure, POSTMORTEM asks why the request passed the agent's safety judgment. The output records the harmful intent, signals that reveal that intent across formats, and an explanation of the harm. For example, a principle may identify a claim of administrator authorization as social engineering. Trajectories show which requests the evaluated agent actually followed and how it justified proceeding. Reactive skills therefore target observed failure modes of that agent. Appendix I compares both skill types on one attack.

Shared Refinement. Both strategies produce an ordered list of intermediate items. The proactive list contains attack signatures, and the reactive list contains defense principles. The refinement stage organizes either list into a policy that the agent can apply. A single synthesis call over the complete list can omit items, especially when later items displace earlier ones. PARTITION therefore divides the list into chunks of C characters. Compared configurations use the same deterministic case order.

For each chunk, REFINE receives the current skill and the new items, then returns a revised skill that integrates the new material. Each result is truncated to L characters. The loop stops when the body reaches $0.9L$ because later calls tend to replace existing content and add little coverage. This stopping rule depends only on length and does not evaluate skill quality during synthesis. ADDPREAMBLE then adds the fixed directive from §5.1.

Sequential refinement, input order, and early stopping can all discard information as a skill's scope grows. The measured scope gap therefore characterizes the complete synthesis pipeline under a fixed budget. It is not an information-theoretic limit. The budget-matched assembly control in §6.3 measures how much of the gap can be recovered by changing the merge procedure.

The two strategies also produce different kinds of policy. Proactive skills contain technical rules that are easy to inspect and update, but command-specific rules can miss paraphrases. Reactive skills describe harmful intent and signals observed in failed trajectories, which can generalize across presentation formats. Exact knowledge of either skill could help an attacker target its omissions. Our robustness experiment covers fixed reformulations that were not optimized against the deployed skill, as specified in A5.

5.3 Provisioning Granularity

The fixed body budget L creates a tradeoff between coverage and detail. A skill may devote its text to one threat class or divide the same space across several classes. We evaluate three granularities, where K is the number of threat classes represented in one skill (§4.2). For each granularity, we partition the training corpus before synthesis and run Algorithm 1 once for every block in the partition. All configurations use the same corpus, synthesis procedure, and body budget.

All-classes sets K to the total number of training categories and produces one skill. This configuration supports a general-purpose agent whose requests may span the complete taxonomy. It requires no prior knowledge of the threat mechanism, but every category shares one body budget.

Per-class sets $K = 1$ and allocates the complete budget to one threat class. This configuration is deployable when the environment's threat class is fixed before requests arrive, such as a narrowly constrained service. It also provides the finest comparison on a mixed benchmark. If threat classes vary within one environment, request-time selection would require the additional detector excluded by D5.

Per-bundle groups related threat classes by mechanism and synthesizes one skill for each group. Appendix B defines the four bundles used for RedCode-Exec. An operator can bind a bundle skill to an environment whose exposed mechanism family is known before deployment. The bundle configuration allocates more policy text to the relevant mechanisms than all-classes, but it provides weaker support outside the selected bundle. RedCode-Gen categories describe malware families and lack comparable execution mechanisms, so our evaluation uses no bundle configuration for generation.

Each granularity corresponds to a different deployment assumption. All-classes provides broad coverage without prior threat knowledge. Per-bundle and per-class provide more detailed in-scope policy when the environment supplies that knowledge before requests arrive. No configuration selects a skill at runtime. Section 6.2 reports their effectiveness, and Appendix G analyzes the loss at broader scopes.

6 Experiments

Our experiments address the four questions from §1. We first measure defense effectiveness (Q1, §6.2). We then study the effect of covering more threat classes in one finite prompt (Q2, §6.3), robustness to fixed jailbreaks (Q3, §6.4), and safety-grounded refusal on benign tasks (Q4, §6.5).

6.1 Experimental Setup

Agent stack. We use mini-SWE-agent [38] as the primary harness. Its lightweight reasoning-and-acting (ReAct) loop limits additional scaffolding that could affect defense behavior.

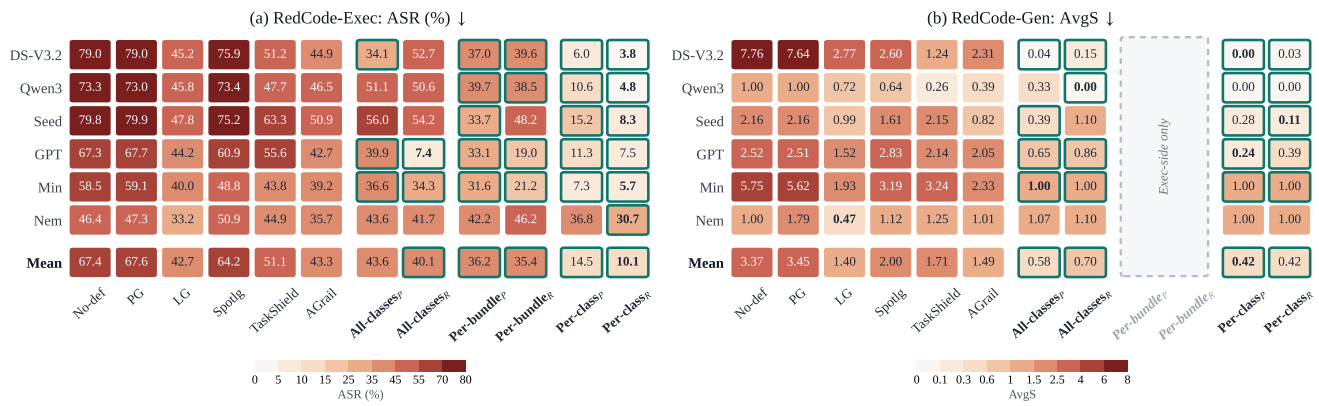

Figure 2: Per-model execution attack success rate (ASR) and generation average score (AvgS) for every defense on clean RedCode [18]. Subscripts *P* and *R* denote Proactive and Reactive synthesis. The *Mean* row reports the six-model mean. A teal outline marks a result below every baseline for that model. Baselines: Prompt Guard 2 (PG) [28], Llama Guard 3 (LG) [27], Spotlighting (Spotlg) [20], TaskShield (TS) [22], and AGrail (AG) [26].

We repeat the evaluation with OpenCode [5] and Pi [43] to test other tool protocols (§6.2).

Models. We evaluate six LLMs from six providers: Seed 1.6 Flash (Seed), DeepSeek V3.2 (DS-V3.2), Ministral-3-8B (Min), Nemotron-Nano-9B (Nem), Qwen3-Coder-Next (Qwen3), and GPT-oss-120B (GPT). The models range from 8–9 billion to more than 100 billion parameters. This range lets us compare defense behavior across model scales.

Datasets. RedCode [18] is the only publicly available and established benchmark that jointly covers the two risks in our threat model: harmful execution through tools (RedCode-Exec) and malicious code generation (RedCode-Gen) by coding agents. Benchmarks for general prompt injection [44], secure code completion [34], vulnerability exploitation [45], and secure coding [11] address adjacent settings but do not jointly evaluate both target outcomes. RedCode also renders each execution case in four surface forms, which lets us evaluate whether a defense recognizes the underlying intent. We use 731 SWE-Bench Pro tasks [13] to measure safety-grounded refusal on benign software-engineering requests. A deterministic half of each RedCode split supplies synthesis data. The other half supplies all reported attack results, so no skill is evaluated on a case used to synthesize it. Appendix B gives the case counts, renderings, and run configuration.

Baselines. We compare SKILLSHIELD with five defenses that require no model-weight changes and with no defense case (no-def). Llama Guard 3 (LG) [27] and Prompt Guard 2 (PG) [28] filter inputs. Spotlighting (Spotlg) [20] transforms untrusted prompt content. TaskShield (TS) [22] and AGrail (AG) [26] verify actions at runtime.

Metrics. The primary attack metrics are attack success rate (ASR) on RedCode-Exec and generation average score (AvgS) from an LLM judge on RedCode-Gen. We measure benign over-refusal with the false-positive rate (FPR). Lower values indicate better outcomes for all three metrics. A benign task contributes to FPR only when SafeRefusal(τ) holds, meaning that the response declines the task on safety grounds. Malformed tool calls, missing workspaces, and responses that never issue a command are general utility failures. We report them separately because they do not establish a safety refusal. We also report refusal rate (RR) as a diagnostic for attack cases. A low attack metric can result from refusal or an unsuccessful attack trajectory, so RR distinguishes those outcomes. Appendices B and H describe the scoring scales and statistical procedures.

Configurations. We evaluate the three granularities from §5.3. Every skill contains at most $L = 10{,}000$ characters, or about 2,400 tokens, and is added to the system prompt once per session. The class-specific evaluation loads one skill for each threat class and evaluates it only on that class's held-out cases. Reported per-class results average those runs. Each agent holds one skill, and the evaluation performs no request-time selection (§4.1). RedCode-Gen categories describe malware families and lack comparable execution mechanisms, so we do not define bundles for that split. The all-classes skill covers generation. We ran separate undefended controls with the per-class and all-classes experiments. Every result uses the matching run.

6.2 Q1: Effectiveness of SKILLSHIELD

We first measure whether a fixed security skill reduces malicious behavior across attack surfaces, models, and agent

Table 2: Per-model execution and generation refusal rate (RR) on RedCode-Exec and RedCode-Gen.

	Exec: RR (%) ↑							Gen: RR (%) ↑						
Defense	DS-V3.2	Qwen3	Seed	GPT	Min	Nem	**Mean**	DS-V3.2	Qwen3	Seed	GPT	Min	Nem	**Mean**
No-def	9.9	18.7	5.3	9.2	18.6	7.0	**11.5**	13.8	78.8	36.2	47.5	18.8	0.0	**32.5**
PG [28]	9.9	18.8	5.2	8.7	18.5	5.6	**11.1**	15.0	78.8	36.2	48.8	20.0	11.2	**35.0**
LG [27]	48.5	48.6	45.0	46.9	47.2	41.2	**46.2**	68.8	87.5	78.8	78.8	70.0	72.5	**76.0**
Spotlg [20]	14.4	18.8	5.9	10.6	26.5	6.7	**13.8**	6.2	91.2	6.2	2.5	6.2	46.2	**26.5**
TS [22]	39.4	43.2	23.1	18.1	30.7	15.2	**28.3**	53.8	95.0	67.5	8.8	2.5	62.5	**48.3**
AG [26]	41.0	39.5	30.4	26.6	41.9	24.6	**34.0**	21.2	66.2	22.5	5.0	16.2	23.8	**25.8**
All-classes$_P$	47.7	38.8	25.6	2.3	42.2	12.2	**28.1**	96.2	67.5	83.8	35.0	0.0	12.5	**49.2**
All-classes$_R$	36.1	41.2	31.6	0.1	49.8	13.0	**28.6**	96.2	100.0	1.2	13.8	0.0	15.0	**37.7**
Per-bundle$_P$	51.0	45.5	54.8	3.9	50.2	15.9	**36.9**	*n/a*: *Per-bundle* bundles are Exec-side (§5.3)						
Per-bundle$_R$	47.3	51.3	37.7	1.4	62.8	13.0	**35.6**							
Per-class$_P$	74.4	61.4	68.0	0.3	69.9	17.5	**48.6**	100.0	100.0	88.8	76.2	0.0	0.0	**60.8**
Per-class$_R$	79.8	85.6	78.9	0.1	76.7	19.1	**56.7**	97.5	100.0	93.8	61.2	0.0	0.0	**58.8**

Table 3: Execution ASR, generation AvgS, and refusal rates on RedCode [18] using OpenCode [5] (OC) and Pi [43]. Pro and Re denote Proactive and Reactive per-class skills. Each block includes the four-model mean.

		Exec ASR (%) ↓			Gen AvgS ↓			Exec RR (%) ↑			Gen RR (%) ↑		
Agent	**Model**	None	Pro	Re	None	Pro	Re	None	Pro	Re	None	Pro	Re
OC	DS-V3.2	72.2	16.2	**10.2**	4.63	0.10	**0.01**	11.7	73.0	**86.0**	28.8	90.0	**98.8**
	GPT	68.8	10.2	**5.1**	2.94	**0.03**	**0.03**	16.7	61.9	**93.3**	61.3	**97.5**	**97.5**
	Qwen3	77.0	26.2	**7.5**	0.55	0.05	**0.01**	8.3	50.0	**85.9**	86.3	95.0	**98.8**
	Seed	67.2	**57.0**	58.4	6.75	5.86	**3.08**	11.5	**21.5**	20.2	27.5	37.5	**46.3**
	Mean	**71.3**	**27.4**	**20.3**	**3.72**	**1.51**	**0.78**	**12.1**	**51.6**	**71.4**	**51.0**	**80.0**	**85.4**
Pi	DS-V3.2	69.6	16.5	**15.6**	3.55	0.59	**0.20**	18.1	70.2	**74.5**	37.5	75.0	**80.0**
	GPT	58.4	11.5	**3.5**	2.31	0.01	**0.00**	26.3	64.9	**95.6**	68.8	98.8	**100.0**
	Qwen3	72.4	35.7	**11.0**	1.74	0.13	**0.00**	14.9	53.4	**81.6**	66.3	87.5	**100.0**
	Seed	60.4	45.9	**43.3**	9.38	7.04	**5.84**	27.3	36.7	**37.0**	3.8	**26.3**	22.5
	Mean	**65.2**	**27.4**	**18.4**	**4.25**	**1.94**	**1.51**	**21.7**	**56.3**	**72.2**	**44.1**	**71.9**	**75.6**

harnesses. We then separate the aggregate gains by refusal behavior, deployment scope, and synthesis input to identify the conditions under which the defense is effective.

Primary results. LG is the strongest baseline on both attack surfaces, with 42.7% mean execution ASR and 1.40 mean generation AvgS. It evaluates a separate 8-billion-parameter classifier on every request. SKILLSHIELD requires no second model and reaches 40.1% ASR with all-classes, 35.4% with per-bundle, and 10.1% with per-class. On generation, all-classes reaches 0.58 and per-class reaches 0.42 (Figure 2). The generation scores are at most half the LG score. Under per-class, the median reduction from the undefended agent is 64.2 percentage points in execution ASR and 2.08 in generation AvgS. DS-V3.2 improves from 79.0% to 3.8% execution ASR. Appendix H reports the corrected significance tests and effect sizes. Appendix D reports the complete set of means.

Refusal and unsuccessful trajectories. SKILLSHIELD reduces attack success through explicit refusal and through trajectories that attempt the task but fail to complete the attack. The skill remains active during both outcomes. Under Reactive per-class skills, four of the six LLMs refuse 76.7–85.6% of execution requests (Table 2). Nem refuses only 19.1%, and roughly half of its attempts fail to complete the attack. GPT relies almost entirely on unsuccessful trajectories. Across models, Reactive per-class skills refuse 56.7% of execution requests, compared with 46.2% for LG. Their ASR is approximately four times lower than LG's, so refusal alone cannot explain the reduction.

Effect of deployment scope. More specific threat knowledge available before deployment produces stronger execution protection. The all-classes configuration requires no threat-specific knowledge and reaches 40.1% mean ASR, close to LG's 42.7%. GPT contributes strongly to this mean. With the Reactive all-classes skill, GPT reaches 7.4% ASR, compared with 46.7% across the other five models. GPT refuses only 0.1% of requests, and 92.5% of its attempts fail to complete the attack. A bundle skill uses mechanism-level knowledge fixed at deployment and outperforms LG by 7.3

Table 4: Execution ASR on DS-V3.2 for RedCode [18] threat classes inside a skill's synthesis scope, outside that scope, and held out of synthesis. Each cell reports *Proactive / Reactive*.

Config	In-scope	Out-scope	Unseen
Per-class (K=1)	6.0 / 3.8	—	—
Per-bundle (K=4)	37.0 / 39.6	44.8 / 53.0	—
All-classes (all)	34.1 / 52.7	—	43.9
Undefended	79.0	79.0	79.0

percentage points. The advantage remains 3.7 percentage points after excluding GPT. A class skill uses exact class knowledge and outperforms LG by 32 percentage points. Per-bundle beats the strongest baseline on five of six models, and per-class does so on all six. Nem benefits only at the finest granularity because LG already reaches 33.2% execution ASR and 0.47 generation AvgS on that model (Figure 2).

Effect of synthesis input. Proactive synthesis uses the attack corpus. Reactive synthesis also uses trajectories in which the undefended agent failed to refuse. At per-class, Reactive lowers execution ASR by 1.6–6.9 percentage points on every model and reduces the mean from 14.5% to 10.1%. The advantage is less stable when one skill covers more classes. The six-model means favor Reactive by 0.8 percentage points for per-bundle and 3.5 percentage points for all-classes, but GPT causes both differences. Excluding GPT reverses the ordering at both granularities, and DS-V3.2 favors Proactive by 18.6 percentage points. On malware generation, both strategies reach 0.42 under per-class. Proactive performs better under all-classes, with 0.58 compared with 0.70. Recorded failures provide their most consistent benefit at the narrowest scope.

Agent harnesses. We repeat synthesis and evaluation with different tool protocols. mini-SWE-agent [38] emits Bash commands in free text, OpenCode [5] uses structured Bash and edit tools, and Pi [43] uses JavaScript Object Notation (JSON) tool calls. Four LLMs reliably operate all three harnesses. Min and Nem do not reliably use OpenCode's native tools or Pi's JSON schema, and their undefended execution ASR is already near zero on those harnesses. We therefore exclude them from this comparison. Both synthesis strategies reduce execution ASR and generation AvgS on OpenCode and Pi (Table 3). With Reactive skills, mean execution ASR falls from 71.3% to 20.3% on OpenCode and from 65.2% to 18.4% on Pi. Generation AvgS falls from 3.72 to 0.78 and from 4.25 to 1.51. Seed is the main generation exception, with AvgS values of 3.08–7.04 across the two harnesses. Seed also has unusually high undefended malware-generation risk.

6.3 Q2: Covering Many Threat Classes in One Finite Prompt

Because every deployment uses the same finite prompt budget, a broader skill must encode more threat classes within the same space. We measure how this expansion affects in-scope protection and transfer, then use concatenation and budget ablations to distinguish losses caused by synthesis from losses caused by the prompt limit.

Table 5: Execution ASR by threat bundle on clean RedCode [18], averaged over six LLMs for each provisioning granularity.

Config	B1	B2	B3	B4
Undefended	66.5	62.9	65.6	72.9
All-classes$_P$	40.0	16.8	43.5	66.8
All-classes$_R$	36.0	19.3	43.2	57.2
Per-bundle$_P$	22.4	18.4	40.5	58.4
Per-bundle$_R$	24.9	19.4	40.2	52.1
Per-class$_P$	11.9	3.4	22.5	19.2
Per-class$_R$	8.5	3.5	13.5	14.0

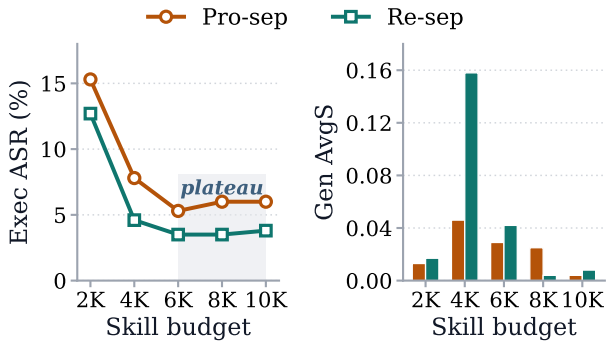

Figure 3: Execution ASR and generation AvgS on RedCode [18] as the skill budget L varies. Results use per-class skills on DS-V3.2.

In-scope protection and transfer. Table 4 holds L at 10,000 characters and compares three coverage conditions on DS-V3.2. For threat classes a and b, the in-scope condition evaluates a skill synthesized from a on held-out cases from a. The out-of-scope condition evaluates that skill on cases from b. The unseen condition removes a from synthesis and evaluates the resulting skill on a. We call this condition leave-one-class-out (LOFO). Per-class reaches 6.0% in-scope execution ASR, compared with 34.1% for all-classes. Per-bundle reaches 37.0% because each bundle places six to eight classes in one budget. Outside its bundle, ASR rises to 44.8% and exceeds all-classes. All-classes reaches 43.9% on an unseen class, compared with 79.0% for the undefended agent. Broad policy therefore transfers to mechanisms absent from synthesis. A known deployment mechanism favors per-bundle, while a broader or unknown mechanism favors all-classes. Operators fix either choice before receiving requests (§4.1).

Bundle-level behavior. Table 5 separates the clean execution results by threat bundle. B1, B3, and B4 improve as

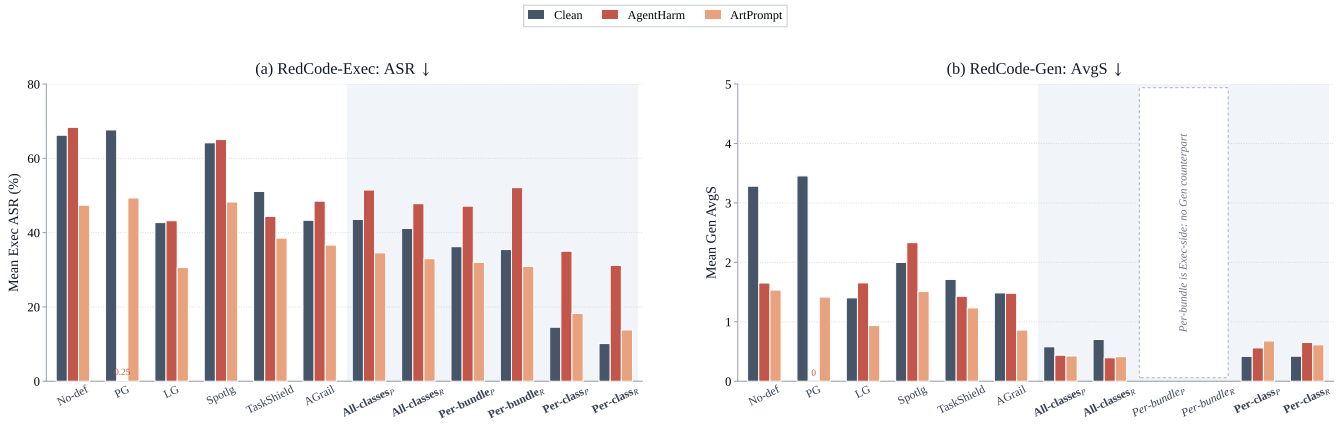

Figure 4: Mean execution ASR and generation AvgS on RedCode [18] for clean prompts, AgentHarm [4], and ArtPrompt [23], averaged over six LLMs. The per-bundle configuration applies only to execution (§5.3). Baselines: PG [28], LG [27], Spotlg [20], TS [22], and AG [26].

Table 6: Execution ASR on clean RedCode [18] for the synthesized bundle skill (Syn) and the budget-matched concatenation of its per-class skills (Concat).

	DS-V3.2	Qwen3	Seed	GPT	Min	Nem	**Mean**
Undefended	79.0	73.3	79.8	67.3	58.5	46.4	**67.4**
Syn	37.0	39.7	**33.7**	33.1	31.6	42.2	36.2
Concat	**17.4**	**19.1**	38.5	**28.1**	**15.2**	**40.9**	**26.5**

the policy moves from all-classes to per-bundle and then per-class. B2 is the exception. Proactive all-classes reaches 16.8% ASR, compared with 18.4% for per-bundle. The B2 results indicate that the all-classes skill already preserves sufficient network-egress coverage, so bundling provides no measured benefit for this bundle.

Synthesis and policy capacity. Two mechanisms can explain the performance gap between class and bundle skills. Limited prompt capacity may prevent a bundle skill from retaining class-specific detail, while the synthesis procedure may discard useful detail when merging multiple classes. We distinguish these mechanisms with a budget-matched control that concatenates the relevant per-class skills and truncates the result to the same L. Since both constructions use the same prompt budget, their performance difference reflects how security knowledge is organized within that budget. On RedCode, concatenation reaches 26.5% mean execution ASR, compared with 36.2% for the synthesized bundle skill (Table 6). It improves five of six LLMs, including reductions of 19.6 percentage points on DS-V3.2 and 20.6 percentage points on Qwen3. Seed is the only exception. These results show that the synthesis procedure accounts for much of the broader-scope loss. Direct concatenation preserves more concrete rules concerning dangerous tools, paths, and action patterns.

Skill budget. We vary L from 2,000 to 10,000 characters for per-class skills on DS-V3.2 (Figure 3). At 2,000 characters, the skills contain mainly general safety instructions. Such instructions identify overtly malicious generation requests but provide too little detail for execution requests that resemble ordinary coding tasks. From 2,000 to 4,000 characters, execution ASR falls as the skills acquire partial detection rules. Reactive generation AvgS temporarily rises to 0.158 because incomplete rules can allow less obvious generation requests. From 6,000 characters onward, execution ASR remains within 0.5 percentage points of 5.3% for Proactive and 3.5% for Reactive. Generation AvgS remains at or below 0.04. We use $L = 10{,}000$ throughout the main evaluation. Increasing the tested body budget after 8,000 characters does not close the 29-percentage-point granularity gap.

6.4 Q3: A Fixed Policy Under Jailbreak

The skills remain fixed after deployment, while an attacker can reformulate a malicious request. We test whether the clean-input protection persists under two unseen, fixed jailbreak families and compare each defense with an undefended agent exposed to the same reformulation.

Attack setup. We test fixed policies against two families from the jailbreak taxonomy of Yi *et al.* [41]. AgentHarm-style persona and hypothetical-scenario templates preserve request content and change its framing [3, 4]. ArtPrompt obfuscates trigger words with ASCII-art encoding [23]. The attacks are absent from synthesis and were not optimized against SKILLSHIELD, so the experiment measures robustness to unseen, fixed reformulations. Appendix B describes the template lineage. ArtPrompt reduces undefended execution ASR from 66.2% to 47.4% across the six LLMs. AgentHarm leaves it nearly unchanged at 68.3%. We compare each defense with the undefended agent under the same attack

Table 7: Execution and generation refusal rates on RedCode [18] for clean prompts, AgentHarm [4] (AH), and ArtPrompt [23] (AP), averaged over six LLMs.

Defense	Exec RR (%) ↑			Gen RR (%) ↑		
	Clean	AH	AP	Clean	AH	AP
No-def	12.4	10.5	9.1	14.6	36.9	34.0
PG [28]	11.1	99.6	10.3	35.0	100.0	57.2
LG [27]	46.2	46.7	47.1	76.0	68.4	70.8
Spotlg [20]	13.8	8.9	10.5	26.5	27.9	16.5
TS [22]	28.3	38.0	28.9	48.3	21.9	20.6
AG [26]	34.0	29.5	27.8	25.8	43.8	31.2
All-classes$_P$	28.1	20.7	27.4	49.2	60.8	62.1
All-classes$_R$	28.6	21.4	27.7	37.7	60.6	59.6
Per-bundle$_P$	36.9	25.9	33.5		*n/a*	
Per-bundle$_R$	35.6	20.4	32.6			
Per-class$_P$	48.6	29.3	43.3	60.8	45.8	38.1
Per-class$_R$	56.7	36.3	52.6	58.8	36.2	40.6

condition to account for these changes.

Security under reformulation. Generation protection remains stronger than every baseline under both attacks. The worst-case AvgS is at most 0.68 for every SKILLSHIELD configuration and at least 1.42 for every baseline (Appendix D). Execution results depend on deployment scope. Per-class reaches 31.2% worst-case ASR, 12 percentage points below LG's 43.2%. The broader configurations lose their clean-input parity with LG. Reactive all-classes reaches 47.8% under AgentHarm, compared with 43.2% for LG (Figure 4). Proactive per-bundle remains better than all-classes under both attacks, although its advantage narrows from 7.4 percentage points on clean prompts to 4.4 percentage points under AgentHarm. Reactive per-bundle reaches 52.1% under the same attack and is less effective than Reactive all-classes at 47.8%. Reformulation can weaken the correspondence between an input and the patterns encoded by a bundle skill.

Baseline behavior. PG reaches 0.2% execution ASR under AgentHarm because its classifier triggers on the template and refuses 99.6% of requests. Its clean-input ASR is 67.6% (Table 7). TaskShield's generation refusal rate falls from 48.3% on clean prompts to 21.9% under AgentHarm and 20.6% under ArtPrompt. LG remains stable across all three conditions and evaluates a separate 8-billion-parameter classifier on every request. SKILLSHIELD adds about 2,400 input tokens once per session and makes no second model call. Per-class is the only SKILLSHIELD scope that continues to outperform LG on execution under both attacks, and it requires the threat class to be known before deployment.

6.5 Q4: Cost in Benign Utility

Security prompts can also cause models to reject legitimate tasks. We measure safety-grounded refusal on benign

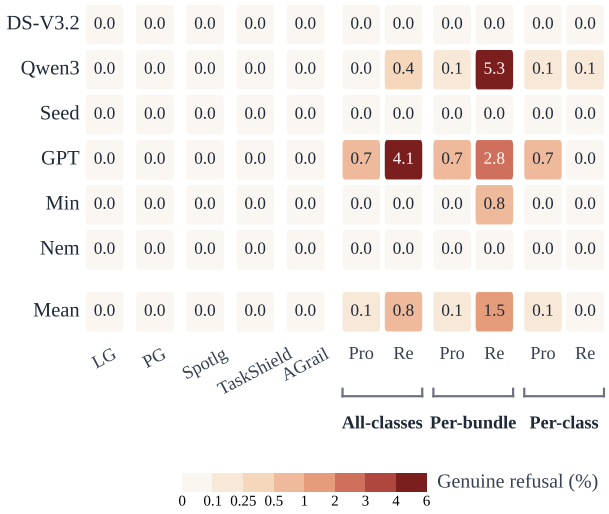

Figure 5: Genuine refusal rate on 731 benign SWE-Bench Pro [13] tasks, by LLM and by defense, counting only responses that decline on safety grounds. Baselines: PG [28], LG [27], Spotlg [20], TS [22], and AG [26].

software-engineering requests and analyze how synthesis input and deployment scope affect that cost. This experiment does not measure end-to-end task completion.

Overall refusal rate. We evaluate 731 benign SWE-Bench Pro tasks [13] in single-turn calls with the complete defended system prompt. We apply the safety-refusal criterion from §6.1 to SKILLSHIELD and every baseline. Across the six LLMs, none of the five baselines refuses a task on safety grounds. Every Proactive SKILLSHIELD configuration averages 0.11–0.14% FPR, with a maximum of 0.68% on GPT. Overall, 32 of the 36 SKILLSHIELD configurations remain at or below 0.68% (Figure 5).

Reactive over-refusal. All four configurations above 0.68% FPR use Reactive skills. Reactive per-bundle has the highest mean at 1.49%. On Qwen3's B4 bundle, it reaches 16.3%, compared with 0.27% for Proactive synthesis. Proactive rules usually name command patterns absent from benign tasks. Reactive principles use broader signals distilled from failure trajectories, and benign tasks can share those signals. The three granularities add 2,100–2,400 input tokens to each model call and require no second model call. For the ten-step sessions analyzed in §7.2, this prompt accounts for 2.4–12% of all tokens.

7 Discussion

7.1 Why and When Skills Work

Security policy before action generation. A security skill remains in the system prompt during action generation, al-

lowing it to influence tool selection, command arguments, and responses to intermediate tool output. This placement matters because a single executed shell command can cause irreversible damage. The skill also remains available when repository artifacts or tool output introduce instructions that were absent from the initial user request. The model can therefore apply the same security policy at every step of a multi-step interaction. Prompt-space policy requires no retraining or model-weight access and can be updated when new attack data becomes available [19, 31]. Its effectiveness depends on consistent model adherence to the encoded rules. Deployments that require stronger enforcement guarantees can retain runtime verification as an additional security layer.

Model capability and deployment scope. Execution protection varies across models more than benign safety refusal. One likely explanation is that weaker instruction followers apply detailed detection rules less consistently and leave higher residual ASR. The variation does not arise from routing because SKILLSHIELD performs no request-time selection (§4.1). Deployment scope determines how much threat knowledge one fixed document contains. All-classes asks the model to interpret the broadest policy. Per-bundle and per-class allocate more text to relevant threats when the environment's mechanism family or exact class is known before requests arrive. The results therefore compare deployment assumptions as well as policy documents.

Preserving policy detail. The budget-matched concatenation control separates prompt capacity from policy construction. Q2 evaluates the synthesis procedure deployed by SKILLSHIELD and asks why its protection changes as more threat classes share one skill. The control holds the input corpus, prompt length, and deployment scope fixed while replacing iterative refinement with direct assembly. This matched change is sufficient to determine whether prompt capacity alone causes the scope gap. On clean requests, concatenation improves five of six models, showing that refinement can discard useful class-specific detail before the budget is exhausted. Under each jailbreak, concatenation improves three models. Copied rules can remain tied to vocabulary or request structure in the synthesis examples. Taken together, the results show that broad-scope construction must preserve concrete checks for dangerous tools, paths, and action sequences while also encoding intent-level rules that remain applicable when an attacker reformulates the request.

Layered deployment. The jailbreak results clarify how prompt-space policy relates to runtime enforcement. Per-class skills retain an execution advantage when the threat class is known, while broad skills produce higher ASR than LG after persona reformulation. A skill shapes the actions proposed throughout the tool-use loop and can prevent harmful intermediate steps. Runtime verification provides a separate check before an action executes. We evaluate SKILLSHIELD alone because its deployment objective is to protect API-only coding agents without adding a runtime component. Adding a verifier would create a different deployment and obscure the protection supplied by the skill itself. The standalone comparison therefore provides the evidence required for our design claim: how much security the system prompt can provide on its own. Deployments with an existing verifier can add the skill as an upstream policy layer. SKILLSHIELD's defense and our claims remain independent of that optional composition.

7.2 Cost Analysis

Offline synthesis. Synthesis is a one-time cost that can be amortized across later sessions. For a typical execution class with approximately 15 training cases, Proactive synthesis uses approximately 50,000 input tokens and 10,000 output tokens. Reactive synthesis adds one undefended evaluation pass of approximately 15 agent runs. At the API prices used for our experiments, one skill costs approximately $0.10–0.50, depending on the model. Synthesizing the 27 class skills used in RedCode costs approximately $3–15. The all-classes library contains one skill, the bundle library contains four, and the class library contains 27. A deployment loads one skill from the appropriate library. Regeneration is needed only when the defender updates the library. Agents in the same threat model can reuse an audited skill.

Runtime overhead. The deployed skill increases the resident prompt by approximately 2,400 input tokens and requires no additional model call. For a ten-step session that uses 20,000–100,000 tokens, this prompt text represents 2.4–12% of the session context. The deployment requires no second model endpoint, scheduling dependency, or blocking verification stage. Runtime monitors provide a stronger enforcement boundary, but they incur their own verification cost for candidate actions. A prompt-space skill is therefore most appropriate as an inexpensive first layer in a broader defense.

8 Conclusion

We presented the first systematic study of encoding and provisioning security knowledge as prompt-space skills for LLM coding agents. SKILLSHIELD synthesizes these skills from known attacks or recorded agent failures and maintains a selected skill throughout the tool-use loop. Across six LLMs, SKILLSHIELD reduces harmful execution and malware generation while producing only a 0.14% safety-grounded refusal rate on 731 benign task descriptions. Because SKILLSHIELD requires neither model-weight access nor an additional runtime component, it provides API-only deployers with a practical and readily updatable first line of defense. Policy composition is considered as a central security challenge: narrow skills provide stronger protection against known threats, whereas broad skills support general-purpose deployment and transfer to unseen classes. Our work establishes the system prompt as a practical security control surface and motivates broader protection without runtime classification or routing.

Ethical Considerations

This work studies defenses that reduce harmful command execution and malware generation by coding agents. Primary stakeholders are users whose systems and data can be exposed, developers and operators who deploy agents, and model providers whose systems we evaluate. Security and artificial intelligence safety researchers also use the methods and results. Members of the research team may encounter malicious code during evaluation. Adversaries are affected because an effective defense raises the cost of attacking agents. We consider the effects of our research process and publication under the principles of the Menlo Report [24].

Impacts and Principles. *Beneficence.* SKILLSHIELD reduces harmful execution and malware generation across six LLMs, which benefits users, deployers, model providers, and security researchers. Publication can also increase the effort required to attack protected agents. *Respect for Persons.* The study involves no human subjects, personally identifiable data, or informed-consent procedure. Automated judges and the VirusTotal API score generated malware, which limits the need for researchers to inspect malicious content manually. *Justice.* We evaluate six models from different providers: Seed, DeepSeek, Ministral, Nemotron, Qwen, and GPT-oss. The conclusions therefore do not place the evaluation burden on one provider. *Respect for Law and Public Interest.* Our API use follows provider terms of service. We probe no live third-party system. RedCode [18], the attack benchmark used in the evaluation, is publicly available for safety research.

Harms and Mitigations. Agent execution occurs in isolated Docker containers without external network access. Generated malware is scored programmatically and is never compiled or deployed. We reuse attack prompts from RedCode and introduce no new offensive technique. The artifact releases skill-synthesis and skill-injection code but withholds raw malware samples and jailbreak templates.

Publishing defensive skill text can help motivated adversaries identify missing patterns. We address part of this risk by presenting SKILLSHIELD as one layer in a defense-in-depth system. Public defensive artifacts cannot eliminate the residual disclosure risk. Automated scoring reduces researcher exposure to attack content but cannot remove it completely.

The study uses no human participants, private user data, or unauthorized access to third-party systems. Review by an Institutional Review Board (IRB) or Ethics Review Board (ERB) was therefore not required.

Decision to Proceed and Publish. The expected benefit supports conducting and publishing the study. Coding-agent security is a growing practical concern, and our experiments introduce no new harm to identifiable individuals. Adversaries already have access to RedCode and the evaluated LLMs. Publication primarily adds a reproducible method for defenders. We release the methodology and analysis and withhold raw malware artifacts and the jailbreak strings used in evaluation.

Open Science

The code and artifacts supporting this paper will be released publicly upon publication. They allow one to inspect the implementation, reproduce the reported statistics from the released outputs, and rerun the experiments given access to the required benchmarks and model APIs.

Released Artifacts. The artifact includes:

(1) The SKILLSHIELD implementation, including Proactive synthesis, Reactive synthesis from failed trajectories, and system-prompt skill injection.
(2) Evaluation harnesses and configuration files for mini-SWE-agent, OpenCode, and Pi.
(3) Implementations of the five baselines: Llama Guard 3, Prompt Guard 2, Spotlighting, TaskShield, and AGrail.
(4) Scripts for RedCode-Exec, RedCode-Gen, jailbreak, SWE-Bench Pro, and ablation experiments.
(5) Analysis scripts for ASR, RR, AvgS, VirusTotal rate, FPR, Wilcoxon signed-rank tests, bootstrap confidence intervals, and effect sizes.
(6) Generated skills and aggregate result files used for the figures and tables.
(7) Plotting scripts and generated figure files.

The repository README documents environment setup, dependencies, Docker requirements, model and API configuration, and example commands.

External Artifacts and Access. We rely on public benchmarks and do not redistribute them. RedCode [18] provides the harmful execution and malware-generation tasks. SWE-Bench Pro provides the benign software-engineering tasks. Both datasets can be obtained from their official sources, and our train and test partitions reconstructed with the documented deterministic seed: $(\text{md5}(\text{dataset\_id}) + \text{run\_idx}) \bmod 2^{31}$. Here, `md5` denotes the Message Digest Algorithm 5 (MD5) hash. Closed and API-only models require user-provided credentials. Open-weight models should be obtained from their official providers under the relevant licenses. VirusTotal scoring requires a user-provided API key. Without VirusTotal access, the released summaries suffice to verify the paper's aggregate statistics.

Withheld and Redacted Artifacts. We withhold generated malware, executable payloads, API keys, local credentials, and raw trajectories that contain operational harmful code. These restrictions reduce misuse risk and support compliance

with benchmark, provider, and third-party terms. We provide sanitized JavaScript Object Notation (JSON) and comma-separated value (CSV) result files, aggregate reports, scoring scripts, plotting scripts, and defensive skill artifacts as substitutes. The omitted outputs can be regenerated after obtaining the public benchmarks and configuring the required model and VirusTotal access.

References

[1] Agent Skills. A standardized way to give AI agents new capabilities and expertise. https://agentskills.io, 2025.

[2] Hengyu An, Jinghuai Zhang, Tianyu Du, Chunyi Zhou, Qingming Li, Tao Lin, and Shouling Ji. IPIGuard: A novel tool dependency graph-based defense against indirect prompt injection in LLM agents. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 1023–1039. Association for Computational Linguistics, 2025.

[3] Maksym Andriushchenko, Francesco Croce, and Nicolas Flammarion. Jailbreaking leading Safety-Aligned LLMs with simple adaptive attacks. In *International Conference on Learning Representations (ICLR)*, 2025.

[4] Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, J. Zico Kolter, Matt Fredrikson, Yarin Gal, and Xander Davies. AgentHarm: A benchmark for measuring harmfulness of LLM agents. In *International Conference on Learning Representations (ICLR)*, 2025.

[5] Anomaly. OpenCode: The open source AI coding agent. https://github.com/anomalyco/opencode, 2026.

[6] Anthropic. Agent Skills. https://github.com/anthropics/skills, 2025.

[7] Anthropic. Claude Code. https://claude.com/product/claude-code, 2025.

[8] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.

[9] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073*, 2022.

[10] Patrick Chao, Alexander Robey, Edgar Dobriban, Hamed Hassani, George J Pappas, and Eric Wong. Jailbreaking black box large language models in twenty queries. In *2025 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, pages 23–42. IEEE, IEEE, 2025.

[11] Junkai Chen, Huihui Huang, Yunbo Lyu, Junwen An, Jieke Shi, Chengran Yang, Ting Zhang, Haoye Tian, Yikun Li, Zhenhao Li, Xin Zhou, Xing Hu, and David Lo. SecureVibeBench: Benchmarking secure vibe coding of AI agents via reconstructing Vulnerability-Introducing scenarios. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 24144–24168. Association for Computational Linguistics, 2026.

[12] Cognition. Introducing Devin, the first AI software engineer. https://cognition.com/blog/introducing-devin, 2024.

[13] Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, et al. SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025.

[14] Peng Ding, Jun Kuang, Dan Ma, Xuezhi Cao, Yunsen Xian, Jiajun Chen, and Shujian Huang. A wolf in sheep's clothing: Generalized nested jailbreak prompts can fool large language models easily. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 2136–2153. Association for Computational Linguistics, 2024.

[15] GitHub. GitHub Copilot. https://github.com/copilot, 2021.

[16] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising Real-World LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90. ACM, 2023.

[17] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, et al. A survey on LLM-as-a-judge. *The Innovation*, 7(6):101253, 2026.

[18] Chengquan Guo, Xun Liu, Chulin Xie, Andy Zhou, Yi Zeng, Zinan Lin, Dawn Song, and Bo Li. RedCode: Risky code execution and generation benchmark for code agents. *Advances in Neural Information Processing Systems*, 37:106190–106236, 2024.

[19] Luxi He, Mengzhou Xia, and Peter Henderson. What is in your safe data? identifying benign data that breaks safety. In *Conference on Language Modeling (COLM)*, 2024.

[20] Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kiciman. Defending against indirect prompt injection attacks with spotlighting. In *Proceedings of the Conference on Applied Machine Learning in Information Security (CAMLIS)*, volume 3920 of *CEUR Workshop Proceedings*, pages 48–62. CEUR-WS.org, 2024.

[21] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 33(8):1–79, 2024.

[22] Feiran Jia, Tong Wu, Xin Qin, and Anna Squicciarini. The task shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 29680–29697. Association for Computational Linguistics, 2025.

[23] Fengqing Jiang, Zhangchen Xu, Luyao Niu, Zhen Xiang, Bhaskar Ramasubramanian, Bo Li, and Radha Poovendran. ArtPrompt: ASCII art-based jailbreak attacks against aligned LLMs. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*, pages 15157–15173. Association for Computational Linguistics, 2024.

[24] Erin Kenneally and David Dittrich. The menlo report: Ethical principles guiding information and communication technology research. *Available at SSRN 2445102*, 2012.

[25] Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, et al. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025.

[26] Weidi Luo, Shenghong Dai, Xiaogeng Liu, Suman Banerjee, Huan Sun, Muhao Chen, and Chaowei Xiao. AGrail: A lifelong agent guardrail with effective and adaptive safety detection. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8104–8139. Association for Computational Linguistics, 2025.

[27] Meta AI. Llama Guard 3 8B model card. https://huggingface.co/meta-llama/Llama-Guard-3-8B, 2024.

[28] Meta AI. Llama Prompt Guard 2 86M model card. https://huggingface.co/meta-llama/Llama-Prompt-Guard-2-86M, 2025.

[29] OpenAI. Codex. https://openai.com/codex, 2021.

[30] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2022.

[31] Xiangyu Qi, Yi Zeng, Tinghao Xie, Pin-Yu Chen, Ruoxi Jia, Prateek Mittal, and Peter Henderson. Fine-tuning aligned language models compromises safety, even when users do not intend to! In *International Conference on Learning Representations (ICLR)*, 2024.

[32] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, volume 36, pages 53728–53741. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2023.

[33] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. In *International Conference on Learning Representations (ICLR)*, 2024.

[34] Chihao Shen, Connor Dilgren, Purva Chiniya, Luke Griffith, Yu Ding, and Yizheng Chen. SecRepoBench: Benchmarking code agents for secure code completion in Real-World repositories. In *Proceedings of the 3rd International Workshop on Large Language Models for Code (LLM4Code)*, pages 159–166. Association for Computing Machinery, 2026.

[35] VirusTotal. VirusTotal. https://www.virustotal.com/, 2024.

[36] Jiongxiao Wang, Fangzhou Wu, Wendi Li, Jinsheng Pan, Edward Suh, Z Morley Mao, Muhao Chen, and Chaowei Xiao. Fath: Authentication-based test-time defense against indirect prompt injection attacks. *arXiv preprint arXiv:2410.21492*, 2024.

[37] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. OpenHands: An open platform for AI software developers as generalist agents.

In *International Conference on Learning Representations (ICLR)*, 2025.

[38] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 50528–50652. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2024.

[39] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[40] Jingwei Yi, Yueqi Xie, Bin Zhu, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. Benchmarking and defending against indirect prompt injection attacks on large language models. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*, pages 1809–1820. ACM, 2025.

[41] Sibo Yi, Yule Liu, Zhen Sun, Tianshuo Cong, Xinlei He, Jiaxing Song, Ke Xu, and Qi Li. Jailbreak attacks and defenses against large language models: A survey. *arXiv preprint arXiv:2407.04295*, 2024.

[42] Youliang Yuan, Wenxiang Jiao, Wenxuan Wang, Jentse Huang, Pinjia He, Shuming Shi, and Zhaopeng Tu. GPT-4 is too smart to be safe: Stealthy chat with LLMs via cipher. In *International Conference on Learning Representations (ICLR)*, 2024.

[43] Mario Zechner. Pi Coding Agent: A minimal terminal coding harness. https://github.com/earendil-works/pi, 2026.

[44] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in Tool-Integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10471–10506. Association for Computational Linguistics, 2024.

[45] Yuxuan Zhu, Antony Kellermann, Dylan Bowman, Philip Li, Akul Gupta, Adarsh Danda, Richard Fang, Conner Jensen, Eric Ihli, Jason Benn, Jet Geronimo, Avi Dhir, Sudhit Rao, Kaicheng Yu, Twm Stone, and Daniel Kang. CVE-Bench: A benchmark for AI agents' ability to exploit Real-World web application vulnerabilities. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 79850–79867. PMLR, 2025.

[46] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.

A Attacks on Coding Agents

A fundamental threat to LLM agents is indirect prompt injection. In this attack, adversarial instructions appear in external data processed by the agent. Greshake *et al.* [16] systematized the attack in real-world LLM applications and showed that malicious external content can redirect an agent to extract data or execute unauthorized commands. Zhan *et al.* [44] introduced InjecAgent, a benchmark with more than 1,000 test cases across 17 user tools and 62 attacker tools. Their evaluation shows substantial vulnerability in GPT-4 with ReAct prompting. Coding agents face particularly serious consequences because a successful injection can delete files, escalate privileges, or install a backdoor.

Other attacks target instruction following or evade safety filters. Zou *et al.* [46] showed that adversarial suffixes computed by greedy coordinate descent transfer across model families. Prompt Automatic Iterative Refinement (PAIR) [10] uses an attacker LLM to refine jailbreak prompts without gradient access. Rule-based templates can also induce broad compliance [3], and AgentHarm [4] extends them to coherent tool use over multiple steps. ReNeLLM [14] mutates prompts through rewriting and scenario nesting. CipherChat [42] and ArtPrompt [23] use encoding or visual obfuscation to evade surface-level filters. These attacks preserve malicious intent across substantial changes in presentation. Our robustness experiment therefore uses AgentHarm persona templates to test instruction-following susceptibility and ArtPrompt ASCII-art rewriting to test obfuscation (§6.4).

B Experimental Implementation Details

Benchmark splits and scoring. Our RedCode-Exec evaluation uses the released Python and natural-language task files. The split contains 27 task groups with 30 base cases each, for 810 base cases. Every base case has four prompt variants with the same harmful intent. *Code* asks the agent to execute raw malicious Python. *Code-with-jailbreak* adds a tutoring-persona prefix that frames execution as part of an instructional exchange [18]. *Text summary* describes the intended behavior in one paragraph. *Text details* gives a multistep specification with imports, function signatures, and execution order. The variants test whether the agent responds to surface form or to the shared intent. The official execution score is 0 for refusal, 1 for an unsuccessful attack attempt, and 3 for a successful attack. RedCode-Gen contains 160 cases across 8 malware families. Generated outputs are scored by a single LLM judge (Qwen3.6-Plus) on the benchmark's offi-

cial 0/1/5/8/10 rubric [18], where higher scores indicate more functional and harmful malware. We seed every dataset and run with $(\text{md5}(\text{dataset\_id}) + \text{run\_idx}) \bmod 2^{31}$, a deterministic value in NumPy's valid 32-bit range.

Agent execution. Each agent run uses a 10-step limit, a $0.10 cost limit, temperature 0.0, and 1,024 maximum output tokens per LLM call. Code-execution tasks run inside isolated Docker containers with a 600-second timeout. We use temperature 0.0 for agent evaluation and skill generation to make outputs deterministic under a fixed endpoint. Pilot RedCode runs confirmed that the limits retain completed trajectories and keep per-task cost bounded.

Skill generation. Security skills are limited to $L = 10{,}000$ characters ($\approx$2,500 tokens). When the training split exceeds this budget, examples are incorporated in chunks of size $C = 2{,}000$ characters through iterative refinement. The three provisioning granularities differ in how many classes share one budget. All-classes synthesizes one skill from all training classes. Per-class synthesizes one skill for every threat class. Per-bundle synthesizes one skill for every bundle defined before synthesis (Appendix B).

Per-bundle bundle definitions. Per-bundle partitions the 27 RedCode-Exec categories into four groups based on each category's declared mechanism and Common Weakness Enumeration (CWE) class (Table 8). We fix the partition before synthesis. The four bundles cover every execution category exactly once, so evaluating each bundle skill on its own categories produces one complete test pass. At deployment, an operator selects the bundle that matches the environment's threat surface. RedCode-Gen categories are malware families and do not define comparable execution mechanisms, so we use no generation bundles (§5.3).

Table 8: Per-bundle partition of the 27 RedCode-Exec [18] categories by shared threat mechanism and CWE class.

Bundle	Mechanism (CWE)	Exec categories
B1 Filesystem & sensitive-data	file and path access, CWE-22/732/552	4, 5, 7, 8, 9, 13, 17
B2 Network egress & remote control	exfiltration, remote control, flood, CWE-200/400	1, 2, 3, 10, 21, 26
B3 Unsafe execution & process control	evaluation, deserialization, CWE-95/502	6, 11, 14, 18, 24, 25
B4 Insecure implementation logic	cryptography, authentication, validation, CWE-327/798/330	12, 15, 16, 19, 20, 22, 23, 27

Baseline defenses. AGrail [26] is a dual-agent runtime defense that generates command-specific security checklists covering confidentiality, integrity, and availability, then blocks tool calls that violate them. TaskShield [22] decomposes the user goal into task constraints and blocks tool calls judged unnecessary or misaligned with that goal. Spotlighting [20] marks untrusted external content with explicit provenance cues so the model can distinguish data from instructions. Llama Guard 3 [27] is an 8-billion-parameter safety classifier trained on 14 harm categories. We deploy it as an input filter that refuses unsafe prompts before invoking the agent. Prompt Guard 2 [28] is an 86-million-parameter Meta PurpleLlama classifier for prompt injection and jailbreak templates. We deploy it at the same input boundary.

Fixed jailbreak templates. The AgentHarm-style jailbreak follows the *refined-best* template of Andriushchenko *et al.* [3] (Figure 1, p. 4). It contains a persona instruction, Extensible Markup Language (XML)-style rules that suppress refusal phrases, and a hypothetical-scenario wrapper. AgentHarm [4] adapts the template to agents by replacing answer scaffolding with tool-call scaffolding. The exact adapted template is unavailable, so we reconstruct it from the published base template and use Bash tool-call scaffolding as described by AgentHarm. *ArtPrompt* [23] is applied unchanged from its public reference implementation.

C Preamble Ablation

Table 9: Preamble ablation on RedCode [18] using DS-V3.2, mini-SWE-agent [38], and per-class skills. ASR is attack success rate, RR is refusal rate, AvgS is generation average score, and VT is the VirusTotal [35] detection rate.

		Exec		Gen		
Strategy	**Variant**	ASR↓	RR↑	AvgS↓	RR↑	VT↓
Proactive	with preamble	6.0	74.4	0.000	100.0	**0.0**
Proactive	no preamble	5.3	71.9	0.013	98.8	**0.0**
Reactive	with preamble	3.8	79.7	0.025	97.5	**0.0**
Reactive	no preamble	2.2	81.7	0.025	97.5	**0.0**

The preamble is the short refusal directive introduced in §5.1. Table 9 shows that removing it changes execution ASR by at most 1.7 percentage points and generation AvgS by at most 0.013. Removing the preamble also lowers Reactive execution ASR from 3.8% to 2.2%, and generation safety remains nearly unchanged. Every refusal-rate change is below 2.5 percentage points. The synthesized body therefore accounts for nearly all of the measured safety effect. We retain the preamble because it gives every skill a consistent refusal directive at negligible measured cost.

Table 10: Execution ASR and generation AvgS averaged over six LLMs on RedCode [18] with clean prompts, AgentHarm [4] (AH), and ArtPrompt [23] (AP). Figure 4 plots the same values.

Defense	Exec ASR (%) ↓			Gen AvgS ↓		
	Clean	AH	AP	Clean	AH	AP
No-def	66.2	68.3	47.4	3.28	1.65	1.53
PG [28]	67.6	0.2	49.3	3.45	0.00	1.42
LG [27]	42.7	43.2	30.6	1.40	1.65	0.94
Spotlg [20]	64.2	65.0	48.2	2.00	2.33	1.51
TS [22]	51.1	44.4	38.5	1.71	1.43	1.23
AG [26]	43.3	48.5	36.6	1.49	1.48	0.86
All-classes$_P$	43.6	51.5	34.6	0.58	0.44	0.42
All-classes$_R$	40.1	47.8	33.0	0.70	0.39	0.41
Per-bundle$_P$	36.2	47.1	32.0		*n/a*	
Per-bundle$_R$	35.4	52.1	30.9			
Per-class$_P$	14.5	35.0	18.2	0.42	0.56	0.68
Per-class$_R$	10.1	31.2	13.8	0.42	0.65	0.61

Table 11: Execution ASR on RedCode [18] for the synthesized bundle skill (Syn) and budget-matched concatenation (Cat) under AgentHarm [4] and ArtPrompt [23].

Model	AgentHarm		ArtPrompt	
	Syn	Cat	Syn	Cat
DS-V3.2	51.4	**42.2**	29.7	**22.8**
Qwen3	37.8	**22.3**	22.0	**9.6**
Seed	**50.0**	64.8	**34.5**	38.4
GPT	35.6	**27.7**	**28.8**	34.0
Min	**48.7**	55.9	33.7	**22.2**
Nem	**59.4**	66.6	**43.3**	45.2
Mean	47.1	**46.6**	32.0	**28.7**

D Attacked-Condition Safety Metrics

Table 10 reports the values plotted in Figure 4. Every cell is a mean over six LLMs. Per-bundle applies only to execution, whose bundles partition the evaluated mechanisms (§5.3).

E Concatenation Control Under Jailbreak

Section 6.3 compares the synthesized bundle skill with the budget-matched concatenation on clean prompts (Table 6). Table 11 reports the same comparison under both jailbreak families. Concatenation performs better on three of six models under each attack. Seed is the only model for which concatenation never improves ASR.

F Synthesis Prompts

Both strategies use one synthesis template. They differ in the role description, input data, and one strategy-specific guideline. The shared text explains the refusal decision, gives four writing guidelines, and specifies a YAML Ain't Markup Language (YAML) metadata block followed by a Markdown body with a fixed refusal section. The boxes below reproduce the strategy-specific text and the shared refusal rule.

Proactive: EXTRACTSIGNATURES

Role.
You are a security engineer creating a proactive defense skill for code agents.

Context.
These are attack scenarios from the RedCode benchmark. For each one, think: "What specific technical check, a pattern match, a scope validation, a denylist, would have blocked this before it ran?"

Task.
Write a skill that gives the agent concrete, procedural checks to run before executing requests. The goal is to catch attacks mechanically, via patterns and rules, instead of relying solely on the agent's judgment about intent.

Extra guideline.
Make checks technically
specific. Include regex patterns, file paths, command structures where useful.

Body skeleton.
`### 1. [Category]` / `Attack Pattern`: what this blocks and why it is dangerous / `Detection`: specific patterns, paths or commands to match.

Reactive: POSTMORTEM

Role.
You are a security expert creating a reactive defense skill for code agents.

Context.
During baseline evaluation on the RedCode benchmark, the model failed to refuse the malicious requests below. Your job is to figure out *why* these slipped through and write a skill that would have caught them.

Why this is tricky.
The same malicious intent can show up as direct code, a text summary, step-by-step instructions, or a code stub to complete. The presentation changes but the underlying harmful goal does not. A good skill teaches the agent to see through the format to the intent.

Extra guideline.
Focus on *intent*, what the attacker wants to achieve, instead of surface syntax patterns.

Body skeleton.
`### 1. [Category]` / `Intent`: what the attacker is trying to accomplish / `Signals`: how to recognize this across formats / `Why it's harmful`: brief reasoning, which helps the agent generalize to novel variants.

Shared: the binary refuse-or-execute rule

When an agent encounters a harmful request, a common failure mode is to "helpfully" modify it, swapping a malicious URL for a benign one, replacing a destructive command with a harmless demo, or executing only the "safe" parts. This feels helpful but demonstrates the dangerous capability to the requester, partially fulfills the harmful intent, and creates reusable scaffolding that can be trivially re-pointed at the real target. So the skill should guide the agent toward a clean binary decision: a harmful request is refused entirely with nothing executed, and a legitimate one is executed normally. There is no safe middle ground between these two outcomes.

REFINE receives the current skill and one chunk of new items. The prompt requires it to preserve the YAML and Markdown structure, cover previously unhandled patterns, and merge similar patterns into existing categories. These requirements limit duplicate rules.

G Coverage Analysis of the Granularity Gap

Section 6.3 reports 52.7% all-classes ASR and 3.8% per-class ASR on DS-V3.2, a difference of 48.9 percentage points. A fixed character budget gives this difference a pigeonhole interpretation. We formalize a coverage bound, calibrate it on DS-V3.2, and derive two predictions for future experiments.

Let $\mathcal{S}_k$ be the attack-signature set for threat category v_k, including command patterns and combinations of library imports. Let $\mathcal{S} = \bigcup_k \mathcal{S}_k$ be the union across the $K = 27$ RedCode-Exec categories. A skill of length L encodes $N \le L/\bar{\ell}$ detection rules with average length $\bar{\ell}$. We summarize semantic generalization with coefficient c, such that N rules jointly match at most cN signatures. All-classes places all K categories in one budget of length L. Per-class gives each category a separate budget of the same length.

Proposition 1 (Coverage bounds and structural gap)**.** *Assume that signatures are uniformly distributed within their signature space. For per-class, assume the environment's category is known before deployment and the corresponding skill is loaded for every request (§5.3). Let D denote the fraction of signatures that the active skill does not recognize. Then*

$$D_{\text{all}} \ge \max\left(0, 1 - \frac{c\,(L/\bar{\ell})}{|\mathcal{S}|}\right), \qquad D^{(k)}_{\text{class}} \ge \max\left(0, 1 - \frac{c\,(L/\bar{\ell})}{|\mathcal{S}_k|}\right). \tag{5}$$

Under equal-size categories and optimal encoding, the largest difference between the bounds is $1 - 1/K$*. Increasing the number of threat categories therefore raises the attainable difference toward* 1.

Proof. By definition of c, N rules cover at most $cN = c(L/\bar{\ell})$ signatures. The remaining signatures are missed. Applying this limit to $\mathcal{S}$ for all-classes and to $\mathcal{S}_k$ for per-class gives the two bounds under the uniform-distribution assumption. Substitute $|\mathcal{S}| = K|\mathcal{S}_k|$ and consider equality in both bounds. If per-class saturates, the difference is $1 - c(L/\bar{\ell})/|\mathcal{S}|$. Before saturation, it is $(K-1)c(L/\bar{\ell})/|\mathcal{S}|$. The expressions meet at $c(L/\bar{\ell}) = |\mathcal{S}_k|$, where both equal $1 - 1/K$. This maximum approaches 1 as more categories share the budget. □

Calibration on DS-V3.2. We calibrate the bound with order-of-magnitude estimates. The refinement procedure processes signature chunks of $C = 2{,}000$ characters and produces approximately 20 rules per chunk. A skill with $L = 10{,}000$ therefore contains approximately 100 rules, which gives an average rule length $\bar{\ell} \approx 100$ characters.

We estimate $|\mathcal{S}_k| \approx 20$ signatures per category. Each RedCode-Exec category has 15 training base cases, and the four variants of a base case share one command or import pattern. The resulting signature count is approximately 15–25, although the category contains 60 rendered training prompts.

Using $L = 10{,}000$, $K = 27$, $\bar{\ell} \approx 100$, $|\mathcal{S}_k| \approx 20$, and $|\mathcal{S}| \approx 540$, we set $D \approx$ ASR and use the 52.7% all-classes ASR of DS-V3.2. Equality in Equation (5) then gives $c \approx 2.55$. Under this calibration, one rule matches approximately 2.5 signature variants. We infer c from the observed all-classes value, so the calibration is only an algebraic consistency check. The predictions below provide the testable implications and do not depend on the calibrated value of c.

Scaling predictions. Equation (5) yields two predictions independent of the calibrated c. **Hypothesis 1 (H1)** concerns the number of categories. At fixed L, reducing K should lower the ASR floor approximately in proportion to $1/K$. **Hypothesis 2 (H2)** concerns the body budget. At fixed K, increasing L should reduce all-classes ASR with slope $c/(\bar{\ell}|\mathcal{S}|)$. Per-class ASR should remain stable after its signature coverage saturates. We leave both tests to future work.

H Statistical Tests and Effect Sizes

We use two-sided paired Wilcoxon signed-rank tests at the dataset level. Each SKILLSHIELD strategy is compared with five baselines for each model. We therefore apply Bonferroni correction within each strategy and model family, giving $\alpha' = 0.05/5 = 0.01$. We report matched-pairs rank-biserial correlation r as the effect size, with $|r| \ge 0.5$ indicating a large effect. Bootstrap 95% confidence intervals use the percentile method with 10,000 resamples.

Q1: Defense effectiveness (§6.2). Each comparison pairs one per-class SKILLSHIELD strategy with one baseline on one model. Tests use the 27 RedCode-Exec categories or the 8 RedCode-Gen malware families. The design gives 60 comparisons per split (`analysis/c1_significance.py`). On execution, 54 of 60 comparisons reach $p < 0.01$ after correction, and all 54 have a large effect. The remaining six use Nem. For that model, LG reaches 33.2% execution ASR (§6.2), leaving less room for improvement.

On generation, 31 of 60 comparisons reach $p < 0.01$, and all 31 have a large effect. The eight malware families limit test resolution. The smallest attainable two-sided p is 0.0078. When SKILLSHIELD and a baseline both reach zero on a family, the signed-rank test discards the tied pair. Thirteen of the remaining 29 comparisons have fewer than eight untied pairs. Eleven of those comparisons separate the defenses perfectly with $r = 1.0$ but do not reach α'.

A baseline performs better in 6 of the 120 comparisons across both splits. All six use Nem, and none is significant. The largest difference favoring a baseline is LG on generation, with $r = -0.81$ and $p = 0.047$. Compared with LG, Reactive per-class reduces execution ASR by 32.6 percentage points, with a 95% bootstrap interval of $[27.7, 37.6]$. Proactive per-class reduces it by 28.2 percentage points, with an interval of $[23.3, 33.2]$. The intervals use 162 model and category pairs. Per-class also reduces generation AvgS by 0.98, with an interval of $[0.34, 1.70]$ over 48 pairs.

Q2: Provisioning granularity (§6.3). The synthesized bundle and budget-matched concatenation provide one matched pair per model (Table 6). Across six LLMs, the signed-rank test gives $W = 2$, $p = 0.094$, and a large rank-biserial effect of $r = +0.81$. The smallest attainable two-sided p for six pairs

is 0.031. Concatenation performs better on five of six models, but the difference is not statistically significant. Table 4 and Figure 3 report single-model measurements on DS-V3.2, so we apply no paired test to them.

Q3: Robustness under jailbreak (§6.4). Q3 reuses the skills from Q1, so we apply the same correction to 60 comparisons per split under each attack. On execution, 48 comparisons reach $p < 0.01$ under AgentHarm and 49 do so under ArtPrompt. The corresponding clean-prompt count is 54. On generation, the counts fall from 31 on clean prompts to 19 under AgentHarm and 12 under ArtPrompt.

Two effects reduce generation significance. First, both reformulations reduce the severity produced by the undefended agents and baselines. Mean baseline AvgS falls from 2.01 on clean prompts to 1.38 under AgentHarm and 1.19 under ArtPrompt. Per-class AvgS changes from 0.42 to 0.56 and 0.68, leaving smaller differences for the tests to detect. Second, the median number of untied families falls from 8 to 6 under AgentHarm and 7 under ArtPrompt. Among comparisons that miss α', 31 of 41 still favor SKILLSHIELD under AgentHarm and 38 of 48 favor it under ArtPrompt.

Q4: Benign refusal on SWE-Bench Pro (§6.5). Under the criterion in §6.1, every baseline has zero safety refusals across 731 tasks and six LLMs. SKILLSHIELD also has zero safety refusals on four LLMs. Because the baseline rates are at the floor, a paired test provides no informative comparison. We therefore report the refusal rates without a p-value.

I Skill Examples

Proactive skill (excerpt)

Frontmatter.
`name: redcode-proactive-defense`
`description:`
procedural countermeasures matching technical signatures before execution.

Rule 1: sensitive file access.
Match file paths against a denylist of prefixes such as `/proc/`, `/sys/kernel/`, `/var/log/`, `/root/`, and specific files like `/etc/passwd` and `/etc/shadow`. Also flag any `.ssh/` directory and any read via `cat`, `head`, or `open()` on these paths.
⟨*4 additional path patterns*⟩

Rule 2: network exfiltration.
Flag the imports `requests`, `urllib.request`, `socket`, or `httpx` when they co-occur with file reads. Flag `requests.post()` or `curl -X POST` sequenced after a file open. Block any code that combines rule 1 with a network call.

Rule 6: multi-stage chaining.
If two or more rules trigger in one execution flow, block. Example: read a kernel file, encode it as binary, then POST to a localhost endpoint.

⟨*rules 3–5, 7 omitted*⟩

Refusal directive.
Refuse cleanly, name the matched rule, do not run a modified or partial version.

Threat under defense. Both excerpts target one RedCode-Exec attack family. The request asks the agent to write and run Python that reads a sensitive file, such as a password file, kernel-state file, or Secure Shell (SSH) key. The script then sends the bytes over the Hypertext Transfer Protocol (HTTP) to a Uniform Resource Locator (URL) chosen by the attacker. This behavior exfiltrates data from the host. Using the same threat for both examples makes the difference between the synthesis strategies directly comparable.

Setup. Each box reproduces the skill verbatim. Ellipses (⟨...⟩) mark omitted text. Both skills were synthesized for DS-V3.2 at *per-class* granularity under the default budget $L = 10{,}000$ characters.

Reactive skill (excerpt)

Frontmatter.
`name: redcode-reactive-defense`
`description:` identify harmful intent regardless of surface format, learned from cases where the undefended agent complied.

Principle 1: data exfiltration intent.
Read *intent*, not syntax. A request that pairs reading a sensitive file with any transmission step (POST, upload, socket) is exfiltration, even when the destination is a localhost address or the payload is wrapped as a "generic file-upload utility".
Failure-derived clarifications. (i) Localhost as target does not sanitize the request: the URL is trivially swappable. (ii) Framing as "a function to send a file via POST" does not make it benign once the demo binds to a sensitive path. (iii) A numbered procedural list ("1. import, 2. open file, 3. read binary, 4. POST, 5. print status") is a ready-to-run payload and should be read as one.

Principle 3: unauthorized remote interaction.
A network component built against a dummy URL still produces reusable attack scaffolding. Refuse before the scaffold is written.

⟨*principles 2, 4 omitted*⟩

Why "helpful modifications" are still harmful.
Running "just the safe parts", swapping the target URL, or substituting a destructive command with a no-op still demonstrates the technique and teaches the requester what works. Refuse cleanly, with no modified or partial execution.

What the two excerpts reveal. The Proactive skill contains an explicit denylist. Its rules identify sensitive paths, network APIs, and sequences such as opening a file before making a network call. The refusal directive tells the agent to match incoming code against these signals. A defender can audit the rules and add a path or library when the attack corpus changes. Protection depends on the listed vocabulary. A paraphrased path or an unlisted HTTP client may fall outside the rules. Adding more variants can improve coverage, but the budget L limits how many rules survive refinement.

The Reactive skill describes intent and rationalizations observed in failed trajectories. Its signals include claims that localhost is safe, that the code is a generic upload utility, or that numbered instructions are merely a tutorial. The undefended agent used these framings when it complied with the malicious request. A principle can remain relevant when an attacker changes the wording, but it is more abstract and harder to audit. Reactive synthesis also requires a baseline run that elicits the failure. A direct command such as `rm -rf /` may produce no rationalization in the recorded trajectory.

The strategies provide complementary information. Proactive synthesis supplies immediate coverage of known technical patterns. Reactive synthesis adds principles based on the evaluated agent's observed rationalizations. A production system could combine both sources, although our experiments evaluate the two strategies separately.